

GUÍA ARQUITECTÓNICA SDD

Asignación de Modelos de IA por Fase y Perfil

 **PREMIUM**
Todos los recursos

 **MIXTO**
Balance inteligente

 **FREE**
Sin costo alguno

Principio Rector

Esta guía aplica el principio de Separación de Responsabilidades a la Inteligencia Artificial. No usamos una Ferrari para ir al supermercado, ni un tractor para una carrera. Cada modelo fue elegido por lo que hace mejor, no por ser el más famoso o el más nuevo.

Capacidad requerida	Tipo de modelo ideal	Ejemplo de fase	Lo que NO usar
Instruction following	Conversacional rápido	ORCHESTRATOR	Modelos de razonamiento (lentos)
Contexto masivo 200K+	Contexto largo especializado	sdd-init	Modelos con ventana de 4K-8K
Razonamiento deductivo	Thinking / Reasoning	sdd-explore / propose / verify	Modelos de código puro
Comprensión visual	Multimodal nativo	sdd-design	Modelos text-only
Generación de código	Fine-tuned en código	sdd-apply	Modelos conversacionales generales
Velocidad pura	Small / Nano models	sdd-tasks / archive	Modelos frontier (desperdicio)



PERFIL PREMIUM

Máxima calidad · Todos los recursos disponibles · Claude + Gemini + ChatGPT Plus + Mistral + OpenCode Go



CONFIGURACIÓN LINEAL

La siguiente tabla muestra el modelo primario y los tres niveles de fallback para cada fase del SDD. El sistema intenta siempre el modelo 1° y cae elegantemente al siguiente si hay rate limit o error.

FASE SDD	1° PRIMARIO	2° FALLBACK	3° FALLBACK	4° FALLBACK
ORCHESTRATOR	claude-sonnet-4-6	gpt-5.3-instant	mistral-large-3	opencode-go/glm-5
sdd-init	opencode-go/kimi-k2.5	gpt-5.3-instant	opencode-go/glm-5	mistral/mistral-nemo
sdd-explore	gpt-5.4-thinking	gpt-5.3-instant	mistral-large-3	mistral/mixtral-8x22b
sdd-propose	gpt-5.4-thinking	gpt-5.3-instant	mistral-large-3	opencode-go/glm-5
sdd-spec	gpt-5.3-instant	mistral/devstral-medium	opencode-go/glm-5	mistral-medium-3.1
sdd-design	gemini-3.1-pro (1 uso)	opencode-go/kimi-k2.5	pixtral-large-latest	gpt-5.4-thinking
sdd-tasks	gpt-5.3-instant	minimax-m2.5	mistral/mistral-nemo	ministral-8b-latest
sdd-apply	mistral/codestral-latest	devstral-2-latest	gpt-5.3-instant	opencode-go/kimi-k2.5
sdd-verify	gpt-5.4-thinking	gpt-5.3-instant	codestral-latest	mistral-large-3
sdd-archive	minimax-m2.5	mimo-v2-omni-free	opencode-zen	big-pickle

JUSTIFICACIÓN TÉCNICA POR FASE

A continuación se explica el razonamiento técnico y estratégico detrás de cada elección, incluyendo por qué cada modelo ocupa su posición específica en la cadena de fallback.

ORCHESTRATOR

1° — claude-sonnet-4-6	El modelo con mayor capacidad de instruction following del mercado. Lee el prompt del usuario, analiza el JSON de herramientas y delega a los sub-agentes sin intentar resolver el problema él mismo. Su fortaleza es obedecer reglas de sistema muy complejas sin desviarse. Se usa 1 SOLA VEZ en todo el ciclo SDD para cuidar los créditos del plugin de navegador.
2° — gpt-5.3-instant	Fallback inmediato. 160 mensajes cada 3 horas, contexto de 128K. Para orquestar se necesita velocidad, no razonamiento profundo. GPT-5.3 Instant no 'piensa' antes de delegar, lo hace al instante, evitando cuellos de botella en el pipeline.
3° — mistral-large-3	El mejor modelo gratuito de Mistral para instruction following y ruteo de herramientas. Si la API de OpenAI está bajo rate limit, Mistral Large 3 mantiene el pipeline activo con 128K de contexto y un nivel de comprensión de sistema muy alto.
4° — opencode-go/glm-5	Paracaídas confiable de OpenCode Go. Muy bueno siguiendo instrucciones de sistema de forma estricta. Ideal para evitar que el orquestador se quede sin opciones si Mistral también está saturado.

sdd-init

1° — opencode-go/kimi-k2.5	La joya del sdd-init. Con 256K tokens de contexto puede ingerir un repositorio completo, el package.json, tsconfig, documentación y requerimientos en una sola llamada sin perder información por el camino. Es el único modelo de la lista que puede 'tragarse' una base de código legacy sin ahogarse.
2° — gpt-5.3-instant	Si Kimi K2.5 está saturado, GPT-5.3 Instant con 128K de contexto entra a leer la documentación inicial. Tiene acceso a todas las herramientas de ChatGPT Plus sin restricción.
3° — opencode-go/glm-5	Según la documentación oficial de OpenCode Go, GLM-5 usa en promedio 52,000 tokens en caché por solicitud. Esto lo hace muy eficiente para leer archivos de configuración que se repiten entre solicitudes.
4° — mistral/mistral-nemo	Modelo gratuito con 128K de contexto y alta velocidad de procesamiento. Si todos los anteriores están bajo rate limit, Nemo puede leer el contexto inicial aunque con menor capacidad total.

sdd-explore

1° — gpt-5.4-thinking	Esta fase requiere razonamiento deductivo: 'si toco el estado del carrito acá, ¿qué componente de la navbar se rompe?' GPT-5.4 Thinking invierte tokens en pensar antes de responder, trazando grafos de dependencias mentalmente. Es el único en la lista que puede explorar implicaciones en cadena sin alucinar.
2° — gpt-5.3-instant	Para exploración de código más directa, GPT-5.3 Instant con 128K es suficiente. Más rápido que Thinking para detectar dependencias obvias, conservando la cuota semanal de Thinking para fases más críticas.
3° — mistral-large-3	El razonamiento analítico de Mistral Large 3 es profundo. Puede seguir dependencias entre módulos y detectar qué código se rompe ante un cambio sin la latencia de los modelos de razonamiento de OpenAI.
4° — mistral/mixtral-8x22b	Modelo Mixture of Experts (MoE). Activa solo los expertos necesarios para el lenguaje que está leyendo: expertos de TypeScript para .ts, expertos de Python para .py. Esto reduce alucinaciones al explorar proyectos multi-lenguaje.

sdd-propose

1° — gpt-5.4-thinking	La fase más crítica del SDD. Proponer si usar Hexagonal vs Clean Architecture, Signals vs Redux, Dependency Injection vs Singleton, requiere evaluar consecuencias a futuro. GPT-5.4 Thinking 'piensa' los trade-offs antes de proponer. Un error acá genera deuda técnica para todo el proyecto.
2° — gpt-5.3-instant	Si se agota la cuota semanal de Thinking, GPT-5.3 Instant puede proponer arquitecturas sólidas aunque con menos profundidad en los trade-offs. Recomendado para features pequeñas o cambios simples.
3° — mistral-large-3	El mejor modelo gratuito para proponer arquitecturas de software. Entiende Hexagonal Architecture, Clean Code, SOLID y los principios de separación de responsabilidades al nivel de un desarrollador Senior.
4° — opencode-go/glm-5	Propone soluciones más estándar y predecibles. No va a proponer algo creativo, pero tampoco va a inventar cosas raras. Es el paracaídas para cuando todo lo demás falla.

sdd-spec

1° — gpt-5.3-instant	Escribir un Markdown técnico de especificación (criterios de aceptación, plan de archivos, contratos de API) requiere seguir un formato estricto y ser preciso con cada sección. GPT-5.3 Instant es excepcional formateando texto largo sin inventar secciones extras.
2° — mistral/devstral-medium	Fine-tuneado específicamente para desarrolladores. A diferencia de los modelos conversacionales que escriben criterios de aceptación vagos, Devstral Medium genera assertions testables, tipos correctos y contratos de API que el programador puede implementar sin ambigüedades.
3° — opencode-go/glm-5	Respetar templates al pie de la letra. Si le das una plantilla de spec con secciones específicas, la va a seguir sin agregar contenido fantasma ni modificar la estructura.
4° — mistral-medium-3.1	Modelo sólido de Mistral para generación de texto estructurado. No es tan orientado a código como Devstral, pero respeta los formatos Markdown con alta fidelidad.

sdd-design

1° — gemini-3.1-pro (1 uso)	El único modelo en el ecosistema con comprensión espacial y multimodal de nivel senior. Puede analizar un wireframe, entender la jerarquía visual de componentes y proponer una estructura de UI en React/Angular con separación correcta entre componentes de presentación y lógica de negocio. Se usa UNA SOLA VEZ para maximizar el impacto de este crédito en la fase donde más brilla.
2° — opencode-go/kimi-k2.5	Tiene encoder de visión nativo MoonViT (400M parámetros). Puede generar código frontend directamente desde mockups o diseños UI en imágenes. Es la mejor alternativa a Gemini Pro cuando el crédito ya fue usado. Con 256K de contexto puede mantener todo el historial de componentes en memoria.
3° — pixtral-large-latest	El modelo multimodal de Mistral, gratuito vía API. Entiende layouts, CSS y la estructura visual de componentes mejor que cualquier modelo text-only. Ideal para proponer un sistema de diseño consistente.
4° — gpt-5.4-thinking	GPT-5.4 mejoró significativamente en generación de código frontend limpio. Como cuarto fallback asegura que el pipeline nunca se quede sin opciones para diseñar la interfaz.

sdd-tasks

1° — gpt-5.3-instant	Dividir una spec en tickets JSON o una lista de TODOs es puro procesamiento de texto sin necesitar razonamiento profundo. GPT-5.3 Instant lo hace en segundos con 128K de contexto, conservando los valiosos mensajes de GPT-5.4 Thinking para fases que los necesitan.
2° — minimax-m2.5	Aproximadamente 30,000 solicitudes cada 5 horas según la documentación oficial de OpenCode Go. Para una tarea tan trivial como dividir tareas, MiniMax M2.5 es prácticamente ilimitado. Nunca habrá rate limit aquí.
3° — mistral/mistral-nemo	128K de contexto y velocidad altísima. Puede leer la spec completa y escupir un JSON de tareas en instantes. Perfecto si los anteriores tienen algún problema puntual.
4° — ministral-8b-latest	Ultra-ligero. Para dividir 10 tareas en subtarear atómicas, no necesitás más que esto. Respeta el principio de usar el mínimo recurso necesario para cada tarea.

sdd-apply

1° — mistral/codestral-latest	El rey del código. Con 256K de contexto puede mantener un repositorio entero en mente mientras escribe. A diferencia de GPT-5.3 que 'explica' el código antes de darlo, Codestral es directo: ejecuta el bloque de código respetando patrones SOLID, convenciones de tipado y arquitectura limpia. Está pre-entrenado en repositorios masivos de GitHub.
2° — devstral-2-latest	El hermano de Codestral, diseñado específicamente para code generation. Si Codestral está bajo rate limit de la API gratuita, Devstral 2 mantiene la calidad arquitectónica del código generado sin introducir acoplamiento.
3° — gpt-5.3-instant	Buen generador de código con 128K de contexto. Es más 'charlatán' que Codestral (explica antes de dar el código), pero con el system prompt correcto puede acotarse a solo generar el bloque necesario.
4° — opencode-go/limi-k2.5	Con 76.8% en SWE-Bench Verified, es el mejor modelo open-source para código. Su 256K de contexto lo hace ideal si necesitás refactorizar archivos muy largos que Codestral rechazaría por context overflow.

sdd-verify

1° — gpt-5.4-thinking	El auditor necesita ser más inteligente que el que escribe. GPT-5.4 Thinking invierte tokens en pensar antes de revisar, detectando vulnerabilidades sutiles, violaciones de SOLID, problemas de lógica de negocio y edge cases que Codestral pasó por alto. Es el 'Senior Reviewer' que no perdona una.
2° — gpt-5.3-instant	Si se agotó la cuota semanal de Thinking, GPT-5.3 Instant puede hacer code review de calidad para bugs evidentes, code smells y desvíos de la spec original. Mejor que no revisar.
3° — codestral-latest	El especialista en código puede auditar el código de sus propios pares. Con el system prompt cambiado a 'modo Senior Reviewer', busca antipatrones, mal uso de tipos y violaciones de la arquitectura propuesta.
4° — mistral-large-3	No es especialista en sintaxis, pero su alto IQ permite detectar problemas de lógica de negocio y mal diseño de contratos en APIs. El paracaídas final del proceso de verificación.

 sdd-archive

1° — minimax-m2.5	150,000 solicitudes por mes según la documentación de OpenCode Go. Resumir logs para Engram es una tarea de compresión de texto que no requiere inteligencia. Usar Claude, Gemini o GPT-5.4 Thinking acá sería un antipatrón financiero. MiniMax M2.5 lo hace de forma prácticamente gratuita e ilimitada.
2° — mimo-v2-omni-free	Modelo gratuito del tier Zen de OpenCode. Rápido y confiable para generar resúmenes cortos de lo que hicieron los sub-agentes. Sin costo, sin rate limit preocupante.
3° — opencode-zen	El modelo nativo de OpenCode Zen. Ligero y siempre disponible en el entorno. No falla para tareas administrativas de guardado en memoria.
4° — big-pickle	El fondo del barril funcional. Si absolutamente todo lo demás falla, Big Pickle puede comprimir un log de texto en un resumen de 2 párrafos. Garantiza que el pipeline nunca se quede sin opciones para cerrar el ciclo.



PERFIL MIXTO

Balance inteligente · Sin Claude ni Gemini Pro · ChatGPT Plus con cuidado · Mistral + OpenCode Go



CONFIGURACIÓN LINEAL

La siguiente tabla muestra el modelo primario y los tres niveles de fallback para cada fase del SDD. El sistema intenta siempre el modelo 1° y cae elegantemente al siguiente si hay rate limit o error.

FASE SDD	1° PRIMARIO	2° FALLBACK	3° FALLBACK	4° FALLBACK
ORCHESTRATOR	gpt-5.3-instant	mistral-large-3	opencode-go/glm-5	qwen3.6-plus-free
sdd-init	opencode-go/kimi-k2.5	qwen3.6-plus-free	mistral/mistral-nemo	mistral/mixtral-8x22b
sdd-explore	mistral-large-3	mistral/mixtral-8x22b	opencode-go/glm-5	nemotron-3-super-free
sdd-propose	gpt-5.4-thinking (1-2 usos)	mistral-large-3	opencode-go/glm-5	qwen3.6-plus-free
sdd-spec	mistral/devstral-medium	opencode-go/glm-5	mistral-medium-3.1	mimo-v2-pro-free
sdd-design	opencode-go/kimi-k2.5	pixtral-large-latest	gpt-5.3-instant	qwen3.6-plus-free
sdd-tasks	minimax-m2.5	mistral/mistral-nemo	nemotron-3-super-free	ministral-8b-latest
sdd-apply	mistral/codestral-latest	devstral-2-latest	opencode-go/kimi-k2.5	qwen3.6-plus-free
sdd-verify	gpt-5.3-instant	mistral-large-3	codestral-latest	nemotron-3-super-free
sdd-archive	minimax-m2.5	mimo-v2-omni-free	opencode-zen	big-pickle

■ JUSTIFICACIÓN TÉCNICA POR FASE

A continuación se explica el razonamiento técnico y estratégico detrás de cada elección, incluyendo por qué cada modelo ocupa su posición específica en la cadena de fallback.

ORCHESTRATOR

1° — gpt-5.3-instant	Sin Claude disponible (reservado para sesiones donde realmente necesitemos su capacidad de tool calling), GPT-5.3 Instant es el mejor orquestador. 160 mensajes cada 3 horas, 128K de contexto, y soporte completo de herramientas. Rápido, no 'piensa' antes de delegar. Perfecto para mantener el flujo del pipeline sin bloqueos.
2° — mistral-large-3	Si OpenAI está bajo rate limit, Mistral Large 3 (API gratuita) toma el control del ruteo. Tiene excelente comprensión de instrucciones de sistema y no se desvía de las reglas definidas en el prompt del orquestador.
3° — opencode-go/glm-5	Backup de OpenCode Go. Muy bueno siguiendo instrucciones estrictas. Si ambas APIs anteriores están saturadas, GLM-5 mantiene el pipeline orquestado correctamente.
4° — qwen3.6-plus-free	Modelo gratuito del tier Zen. Tiene suficiente capacidad de razonamiento para mantener el hilo de la orquestación en emergencias. El paracaídas final.

sdd-init

1° — opencode-go/kimi-k2.5	Igual que en el perfil Premium: 256K de contexto es irremplazable para la ingestión inicial. Con la suscripción de Go a \$10/mes, Kimi K2.5 tiene aproximadamente 9,250 solicitudes por mes. Más que suficiente para el init de múltiples proyectos.
2° — qwen3.6-plus-free	Contexto largo, gratuito. Si Kimi K2.5 alcanza su límite mensual de Go, Qwen cubre la ingestión de repositorios sin costo adicional. El segundo mejor para leer código legacy.
3° — mistral/mistral-nemo	128K de contexto gratis. Rápido para leer archivos de configuración y documentación inicial cuando los modelos con más contexto están temporalmente saturados.
4° — mistral/mixtral-8x22b	MoE con 64K de contexto. Activa los expertos correctos para cada lenguaje, siendo el fallback más robusto para proyectos multi-lenguaje donde los otros modelos fallan.

sdd-explore

1° — mistral-large-3	En el perfil mixto conservamos GPT-5.4 Thinking para propose y verify. Para la exploración, Mistral Large 3 gratuito tiene el razonamiento analítico necesario para detectar qué se rompe si tocamos un módulo. Su contexto de 128K es suficiente para analizar la mayoría de los proyectos.
2° — mistral/mixtral-8x22b	MoE: activa expertos específicos para el lenguaje que analiza. Ideal para detectar dependencias cruzadas entre módulos de diferentes tecnologías (ej. backend TypeScript que llama a un servicio Python).
3° — opencode-go/glm-5	Analítico y estructurado. Puede seguir el rastro de una variable a través de múltiples archivos y mapear las dependencias de forma clara. Buen análisis de impacto sin costo de razonamiento profundo.
4° — nemotron-3-super-free	Sorprendentemente bueno para trazar variables y dependencias entre módulos siendo completamente gratuito. El cuarto fallback que garantiza que la exploración nunca se detenga.

sdd-propose

1° — gpt-5.4-thinking (1-2 usos)	En el perfil mixto, Gemini Pro se reserva para design (donde más brilla) y GPT-5.4 Thinking se invierte aquí. Proponer la arquitectura correcta tiene mayor impacto en la calidad del proyecto que auditar el código. Con 1-2 usos bien colocados en propose, se obtiene el mayor retorno de la inversión de los tokens de Thinking.
2° — mistral-large-3	El mejor gratis para proponer arquitecturas. Si se agotaron los usos de Thinking, Mistral Large 3 puede proponer soluciones de nivel Senior que sigan principios SOLID y arquitecturas limpias.
3° — opencode-go/glm-5	Propone soluciones estructuradas y predecibles. Ideal para features donde la arquitectura no requiere creatividad sino consistencia con el patrón ya establecido en el proyecto.
4° — qwen3.6-plus-free	Propone soluciones muy lógicas y bien organizadas para ser un modelo gratuito. El paracaídas para cuando todo falla: te va a dar una propuesta funcional aunque no sea la más elegante.

sdd-spec

1° — mistral/devstral-medium	Fine-tuneado para devs. Genera specs técnicas con criterios de aceptación testables, no criterios vagos. La diferencia con un modelo conversacional es enorme: Devstral escribe 'debe retornar 404 cuando el usuario no existe', no 'debe manejar errores correctamente'.
2° — opencode-go/glm-5	Respeto el formato de la spec al pie de la letra. Con el template correcto en el system prompt, GLM-5 sigue cada sección sin inventar ni omitir nada. Muy confiable para documentación estructurada.
3° — mistral-medium-3.1	Modelo sólido para texto estructurado. No es tan 'dev-centric' como Devstral, pero produce specs bien formateadas que son fáciles de seguir para el equipo de desarrollo.
4° — mimo-v2-pro-free	Sigue plantillas estrictas sin alucinar secciones. El paracaídas gratuito que asegura que la spec siempre tenga el formato correcto aunque sea el fallback final.

sdd-design

1° — opencode-go/kimi-k2.5	CAMBIO CLAVE del perfil mixto respecto al premium. Kimi K2.5 tiene encoder de visión nativo MoonViT. Puede generar código frontend directamente desde mockups o imágenes de wireframes. Con 256K de contexto mantiene todos los componentes previamente diseñados en memoria para garantizar consistencia visual. Reemplaza a Gemini Pro en este perfil para conservar ese crédito.
2° — pixtral-large-latest	El modelo multimodal de Mistral (gratuito). Entiende layouts, CSS y componentes UI. Si Kimi K2.5 está bajo su límite de Go, Pixtral mantiene la calidad del diseño de frontend sin necesitar modelos de pago.
3° — gpt-5.3-instant	GPT-5.4 mejoró mucho en generación de código frontend limpio. Puede diseñar componentes en React con Tailwind sin la comprensión visual de Kimi o Pixtral, pero con una calidad de código que cumple la spec.
4° — qwen3.6-plus-free	Tiene capacidades de visión básicas y puede diseñar componentes simples. Para interfaces que no requieren lógica visual compleja, es una opción gratuita válida como último recurso.

sdd-tasks

1° — minimax-m2.5	30,000 solicitudes cada 5 horas. Para dividir tareas, es prácticamente ilimitado. En el perfil mixto, preservamos los mensajes de GPT-5.3 para fases más importantes. MiniMax M2.5 hace esta tarea trivial sin tocar las cuotas de OpenAI.
2° — mistral/mistral-nemo	128K de contexto, ultra-rápido. Si MiniMax tiene algún problema puntual, Nemo lee la spec completa y genera los tickets en instantes, completamente gratis.
3° — nemotron-3-super-free	Muy bueno generando JSON estructurado y listas de TODOs. Completamente gratuito y sin rate limits preocupantes para tareas de partición de trabajo.
4° — ministral-8b-latest	Ultra-ligero. El modelo más pequeño de Mistral para tareas simples. Si necesitamos solo dividir 5 tareas en subtareas, Ministral 8B basta perfectamente.

sdd-apply

1° — mistral/codestral-latest	Igual en todos los perfiles: Codestral es el rey indiscutido para escribir código. 256K de contexto, entrenado en repositorios masivos, va directo al código sin explicaciones innecesarias. Respeta la arquitectura propuesta en sdd-propose y las tareas definidas en sdd-tasks.
2° — devstral-2-latest	Si Codestral tiene rate limit en la API gratuita, Devstral 2 toma el relevo. Diseñado específicamente para code generation, mantiene la separación de lógica de negocio y UI establecida en la spec.
3° — opencode-go/kimi-k2.5	76.8% en SWE-Bench Verified. El mejor open-source para código cuando no hay acceso a los modelos de Mistral. Con 256K puede refactorizar archivos muy largos en una sola pasada.
4° — qwen3.6-plus-free	Increíblemente capaz para código siendo gratuito. Escribe funciones limpias aunque puede necesitar un poco más de guía en arquitecturas complejas como Hexagonal. El último rescatista del pipeline de código.

sdd-verify

1° — gpt-5.3-instant	En el perfil mixto, usamos GPT-5.3 para verify en lugar de GPT-5.4 Thinking (que fue usado en propose). Con 128K y su capacidad analítica, puede detectar bugs lógicos, code smells y desvíos de la spec sin consumir la cuota semanal de Thinking.
2° — mistral-large-3	Alto IQ para detectar antipatrones lógicos y problemas de diseño de API. Si OpenAI está bajo rate limit, Mistral Large 3 actúa como el Senior Reviewer gratuito del pipeline.
3° — codestral-latest	Con el system prompt cambiado a 'modo reviewer', Codestral puede auditar código que él mismo podría haber escrito. Conoce los patrones correctos y detecta cuándo el código se desvía de ellos.
4° — nemotron-3-super-free	Verifica que el código cumpla con la spec básica y que no haya errores de sintaxis obvios. El auditor de mínima calidad que garantiza que nada se archive sin al menos una revisión básica.

 sdd-archive

1° — minimax-m2.5	Prácticamente ilimitado y casi gratuito. Para comprimir logs y guardar el contexto en Engram, MiniMax M2.5 hace el trabajo sin ninguna preocupación de rate limit ni costo.
2° — mimo-v2-omni-free	Modelo gratuito de OpenCode Zen. Rápido para generar resúmenes de 2-3 párrafos de lo que ejecutaron los sub-agentes. Disponible siempre que OpenCode Zen esté activo.
3° — opencode-zen	Nativo del entorno, siempre disponible, nunca falla para tareas administrativas. El fallback del fallback gratuito.
4° — big-pickle	El modelo más básico del ecosistema. Para comprimir texto, cumple. Si absolutamente todo falla, Big Pickle guarda el estado en Engram sin gastar ni un centavo.

 PERFIL FREE

100% Gratuito · Solo API gratis de Mistral + OpenCode Zen · Sin tarjeta requerida

 CONFIGURACIÓN LINEAL

La siguiente tabla muestra el modelo primario y los tres niveles de fallback para cada fase del SDD. El sistema intenta siempre el modelo 1° y cae elegantemente al siguiente si hay rate limit o error.

FASE SDD	1° PRIMARIO	2° FALLBACK	3° FALLBACK	4° FALLBACK
ORCHESTRATOR	mistral-large-3	qwen3.6-plus-free	opencode-go/glm-5	mimo-v2-pro-free
sdd-init	qwen3.6-plus-free	opencode-go/kimi-k2.5	mistral/mistral-nemo	mistral/mixtral-8x22b
sdd-explore	mistral/mixtral-8x22b	mistral-large-3	nemotron-3-super-free	qwen3.6-plus-free
sdd-propose	mistral-large-3	opencode-go/glm-5	qwen3.6-plus-free	mistral/mixtral-8x22b
sdd-spec	mistral/devstral-medium	opencode-go/glm-5	mistral-medium-3.1	mimo-v2-pro-free
sdd-design	pixtral-large-latest	opencode-go/kimi-k2.5	mistral/pixtral-12b	qwen3.6-plus-free
sdd-tasks	mistral/mistral-nemo	nemotron-3-super-free	ministral-8b-latest	minimax-m2.5
sdd-apply	mistral/codestral-latest	devstral-2-latest	qwen3.6-plus-free	mistral/mistral-nemo
sdd-verify	mistral/codestral-latest	mistral-large-3	nemotron-3-super-free	qwen3.6-plus-free
sdd-archive	mimo-v2-omni-free	opencode-zen	big-pickle	minimax-m2.5

JUSTIFICACIÓN TÉCNICA POR FASE

A continuación se explica el razonamiento técnico y estratégico detrás de cada elección, incluyendo por qué cada modelo ocupa su posición específica en la cadena de fallback.

ORCHESTRATOR

1° — mistral-large-3	El mejor orquestador 100% gratuito. La API de Mistral La Plateforme ofrece un tier gratuito para experimentación. Mistral Large 3 con 128K de contexto tiene suficiente instruction following para leer el JSON de herramientas y delegar correctamente a los sub-agentes. No es Claude, pero es el más cercano disponible sin costo.
2° — qwen3.6-plus-free	Suficiente razonamiento para mantener el hilo de la orquestación cuando Mistral Large 3 está bajo rate limit de la API gratuita. Contexto amplio y generoso límite de uso.
3° — opencode-go/glm-5	Si tenés la suscripción de OpenCode Go (\$10/mes), GLM-5 es un backup muy confiable para orquestar. Muy bueno siguiendo instrucciones de sistema de forma estricta.
4° — mimo-v2-pro-free	El paracaídas final completamente gratuito. Rápido para despachar tareas al sub-agente correcto en emergencias cuando todo lo demás falla.

sdd-init

1° — qwen3.6-plus-free	El rey del init gratuito. Tiene una ventana de contexto enorme para ser un modelo free y puede ingerir repositorios completos sin cortarse a la mitad. El segundo mejor del ecosistema para leer contexto masivo, superado solo por Kimi K2.5 que requiere suscripción Go.
2° — opencode-go/kimi-k2.5	Si tenés la suscripción Go, Kimi K2.5 con 256K tokens es la opción ideal. La nota aclaratoria es que su costo real viene de Go (\$10/mes), no del tier completamente free.
3° — mistral/mistral-nemo	128K de contexto totalmente gratuito. Rápido para leer archivos de configuración, documentación básica y el contexto inicial de proyectos medianos.
4° — mistral/mixtral-8x22b	MoE con 64K. La arquitectura de Mezcla de Expertos lo hace muy eficiente para leer proyectos multi-lenguaje. Si los anteriores están bajo rate limit, Mixtral garantiza que el init no se quede a medias.

sdd-explore

1° — mistral/mixtral-8x22b	El mejor modelo gratuito para exploración. Su arquitectura MoE activa solo los expertos necesarios para el lenguaje que está analizando. Cuando detecta una dependencia en TypeScript activa sus expertos de TS; cuando cruza a Python, activa otros. Esto reduce alucinaciones al trazar dependencias cruzadas.
2° — mistral-large-3	Razonamiento analítico profundo para explorar qué módulos se ven afectados por un cambio. Detecta dependencias no obvias y puede trazar el impacto de un cambio a través de capas de la arquitectura.
3° — nemotron-3-super-free	Bueno para seguir rastros de variables y mapear qué componentes consumen qué servicios. Gratuito y disponible cuando Mixtral y Mistral Large están bajo rate limit de la API gratis.
4° — qwen3.6-plus-free	Cobertura general. Si los tres anteriores están saturados, Qwen puede hacer una exploración básica del código para no detener el pipeline.

sdd-propose

1° — mistral-large-3	En el perfil free, este es tu arquitecto. Con 128K de contexto y un nivel de razonamiento muy alto para un modelo gratuito, puede proponer arquitecturas sólidas (Clean Architecture, patrones de diseño, separación de capas). No es GPT-5.4 Thinking, pero para proyectos con arquitectura ya definida, cumple muy bien.
2° — opencode-go/glm-5	Estructurado y lógico. Propone soluciones bien organizadas que respetan los patrones existentes en el proyecto. Ideal como segundo en este perfil si tenés la suscripción Go.
3° — qwen3.6-plus-free	Propone soluciones muy estructuradas y lógicas para ser gratuito. En el perfil free es tu tercer arquitecto. Va a proponer algo más estándar que Mistral Large, pero funcional y sin alucinaciones graves.
4° — mistral/mixtral-8x22b	MoE: activa expertos de arquitectura de software para proponer patrones de diseño estándar. El fallback final que garantiza que el proyecto siempre tenga una propuesta arquitectónica, aunque sea conservadora.

sdd-spec

1° — mistral/devstral-medium	Igual en todos los perfiles: Devstral Medium es el especialista en specs técnicas. Fine-tuneado para devs, genera criterios de aceptación testables y planes de archivos que el equipo puede implementar directamente. La API gratuita de Mistral incluye acceso a Devstral.
2° — opencode-go/glm-5	Respeto el formato del template de spec al pie de la letra. Si Devstral tiene rate limit en la API gratuita, GLM-5 (via Go) mantiene la calidad estructural de la especificación.
3° — mistral-medium-3.1	Modelo sólido para texto estructurado en Markdown. No es tan orientado a código como Devstral, pero produce specs bien formateadas que el equipo puede seguir sin ambigüedades.
4° — mimo-v2-pro-free	Sigue plantillas estrictas sin alucinar. La garantía final de que la spec siempre tenga la estructura correcta aunque sea el modelo más básico que la genere.

sdd-design

1° — pixtral-large-latest	El mejor modelo gratuito para UI/UX. La API de Mistral incluye acceso a Pixtral Large (128K, multimodal). Entiende layouts, estructuras de componentes CSS/Tailwind y cómo separar componentes de presentación de los de lógica. En el perfil free, es tu diseñador de interfaces.
2° — opencode-go/kimi-k2.5	Si tenés Go, Kimi K2.5 con su encoder visual MoonViT puede generar código frontend desde diseños. 256K de contexto lo hace ideal para proyectos con muchos componentes interconectados.
3° — mistral/pixtral-12b	El hermano menor de Pixtral Large. También multimodal, menor capacidad pero igualmente gratuito. Entiende layouts y puede diseñar componentes simples de forma consistente.
4° — qwen3.6-plus-free	Tiene capacidades de visión básicas. Para interfaces simples sin lógica visual compleja, puede proponer la estructura de componentes necesaria como último recurso gratuito.

sdd-tasks

1° — mistral/mistral-nemo	El speedster del ecosistema gratuito. 128K de contexto y velocidad altísima para leer una spec completa y generar el JSON de tickets al instante. En el perfil free, es el mejor para esta tarea de procesamiento puro.
2° — nemotron-3-super-free	Muy bueno generando listas y JSON estructurado rápidamente. Si Nemo está bajo rate limit, Nemotron hace la misma tarea sin costo y sin demoras significativas.
3° — ministral-8b-latest	Ultra-ligero y gratuito. Para dividir una spec en tareas atómicas simples, Ministral 8B basta perfectamente. El principio del mínimo recurso necesario.
4° — minimax-m2.5	Si tenés Go, MiniMax M2.5 con sus 30,000 solicitudes cada 5 horas es prácticamente ilimitado para esta tarea trivial. El paracaídas de volumen para el pipeline de tareas.

sdd-apply

1° — mistral/codestral-latest	El rey del código en todos los perfiles. Gratuito vía la API de Mistral La Plateforme, con 256K de contexto. En el perfil free, Codestral es literalmente el mejor modelo disponible para escribir código. No hay ningún modelo de pago que justifique superar a Codestral en code generation puro.
2° — devstral-2-latest	Si Codestral está bajo rate limit de la API gratuita, Devstral 2 asume sin perder calidad. Diseñado específicamente para code generation, es el fallback natural del rey del código.
3° — qwen3.6-plus-free	Excelente para código siendo gratuito. Escribe funciones limpias y respeta convenciones. El tercer coder del pipeline free que garantiza que el código siempre se entregue aunque haya problemas con las APIs de Mistral.
4° — mistral/mistral-nemo	128K de contexto y velocidad alta. No es un modelo especializado en código como Codestral, pero puede escribir funciones simples y hacer cambios menores al código existente. El último fallback para no detener el pipeline.

sdd-verify

1° — mistral/codestral-latest	En el perfil free, el mismo Codestral que escribe el código también lo audita, pero con un system prompt completamente diferente: 'Actúa como Senior Code Reviewer. Busca violaciones de SOLID, code smells, bugs de lógica y desvíos de la spec. NO sugieras cambios, solo reporta problemas con su ubicación exacta'. El experto en código puede auditar al experto en código.
2° — mistral-large-3	Alto IQ para detectar antipatronos de lógica de negocio y mal diseño de contratos de API. Complementario a Codestral: mientras Codestral detecta problemas de sintaxis y patronos, Mistral Large detecta problemas de lógica y arquitectura.
3° — nemotron-3-super-free	Verifica que el código cumpla con la spec básica y que no haya errores obvios. El auditor mínimo que garantiza al menos una revisión de la spec antes de archivar.
4° — qwen3.6-plus-free	Cobertura general de revisión cuando los anteriores están saturados. Puede detectar los problemas más visibles aunque sin la profundidad de Codestral o Mistral Large.

1° — mimo-v2-omni-free	El modelo más adecuado del tier gratuito de OpenCode Zen para comprimir logs. Tarea trivial: tomar el historial de ejecución de los sub-agentes y generar un resumen de 2 párrafos para Engram. Sin costo, sin rate limit preocupante.
2° — opencode-zen	Nativo del entorno OpenCode. Siempre disponible, nunca tiene problemas de autenticación. El fallback más confiable para el cierre del pipeline.
3° — big-pickle	El modelo más básico disponible. Para comprimir texto y guardar el contexto, cumple su función. La garantía final de que el pipeline siempre cierra correctamente.
4° — minimax-m2.5	Si tenés Go, el fallback de mayor volumen. 150,000 solicitudes por mes aseguran que el archivo del contexto nunca falle por rate limit.

Glosario Rápido

Término	Definición
Rate Limit	Límite de solicitudes por unidad de tiempo. Cuando se alcanza, el proveedor devuelve un error 429 y el sistema cae al siguiente fallback.
Graceful Degradation	Caída elegante del sistema. Si el modelo 1° falla, el pipeline automáticamente intenta el 2°, luego el 3° y así sucesivamente sin interrumpir el flujo de trabajo.
MoE (Mixture of Experts)	Arquitectura donde el modelo activa solo los sub-expertos necesarios para cada token, reduciendo latencia y alucinaciones para tareas especializadas.
Fine-tuned	Modelo entrenado adicionalmente sobre una tarea específica (ej. Codestral en código, Devstral en desarrollo). Son más precisos que modelos generalistas para esas tareas.
Ventana de contexto	Cantidad máxima de tokens (palabras/código) que el modelo puede procesar en una sola llamada. 256K $\approx$ 200,000 líneas de código.
SOLID	Principios de diseño de software: Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, Dependency Inversion. El estándar de calidad que verifica sdd-verify.
Engram	Sistema de memoria de OpenCode. sdd-archive guarda el contexto del ciclo SDD en Engram para que el orquestador pueda consultarlo en futuros ciclos.
SWE-Bench	Benchmark estándar de la industria para medir qué tan bien un modelo puede resolver tareas reales de ingeniería de software en GitHub. El gold standard para evaluar coders de IA.